

# Análise Estatística e Controles

## OWASP Top 10



Projeto de Bloco - TP2

*API Customer Support Intent* auditada com OWASP ZAP

**Grupo:** Bernardo de Moraes Eleuterio, Felipe Roberto Rocha,  
Guilherme Valentim Ramalho da Silva e Heitor Cella Oliveira

**Disciplina:** Projeto de Bloco: Análise e Segurança de Agentes de IA

**Professor:** Tiago Cariolano de Souza Xavier

**Instituição:** Instituto Infnet

**Data:** 21 de Setembro de 2026

## Sumário executivo

Este relatório documenta a Etapa 2 do Projeto de Bloco, que teve dois eixos: **aprofundar a análise exploratória** do dataset de chamados de suporte e **transformar a API FastAPI do TP1 em uma API endurecida**, com os controles do OWASP Top 10 aplicados e verificados por auditoria com OWASP ZAP.

### Os dois resultados centrais:

1. **O dataset não sustenta o classificador de intenção planejado.** A maior correlação entre variáveis distintas é  $|r| \approx 0,04$ ; o assunto do chamado é estatisticamente independente do seu tipo (qui-quadrado,  $p = 0,98$ ); e um baseline TF-IDF + Regressão Logística atinge 18,9% de acurácia - *abaixo* do modelo trivial (20,7%) e equivalente ao acaso. A causa é objetiva: 69,3% dos chamados compartilham exatamente o mesmo texto de abertura, rotulado com as cinco intenções em proporções iguais.
2. **Todos os alertas Medium e Low do OWASP ZAP foram eliminados.** O scan passivo da API do TP1 apontou 30 alertas Medium e 17 Low; após os controles implementados, o scan de verificação retorna **zero** alertas Medium, Low e High.

## Entregas por tarefa do enunciado

1 EDA com heatmap, scatter e teste de hipótese

2 Controles OWASP (extra=forbid, SQLAlchemy, BOLA)  
Notebook seções 9–14; §1 deste relatório

models/ , database.py ,  
routes/tickets.py ; §2

3 Headers de segurança e CORS security/middleware.py , config.py ; §3 4 Rate limiting no /auth/token

security/rate\_limit.py ; §4 5 Scan OWASP ZAP e findings security/zap/ , ZAP\_FINDINGS.md ; §5 6 Testes pytest de  
segurança tests/test\_security.py ; §6 7 Relatório de EDA estruturado eda/RELATORIO\_EDA.md ; §1

# 1. Análise exploratória aprofundada

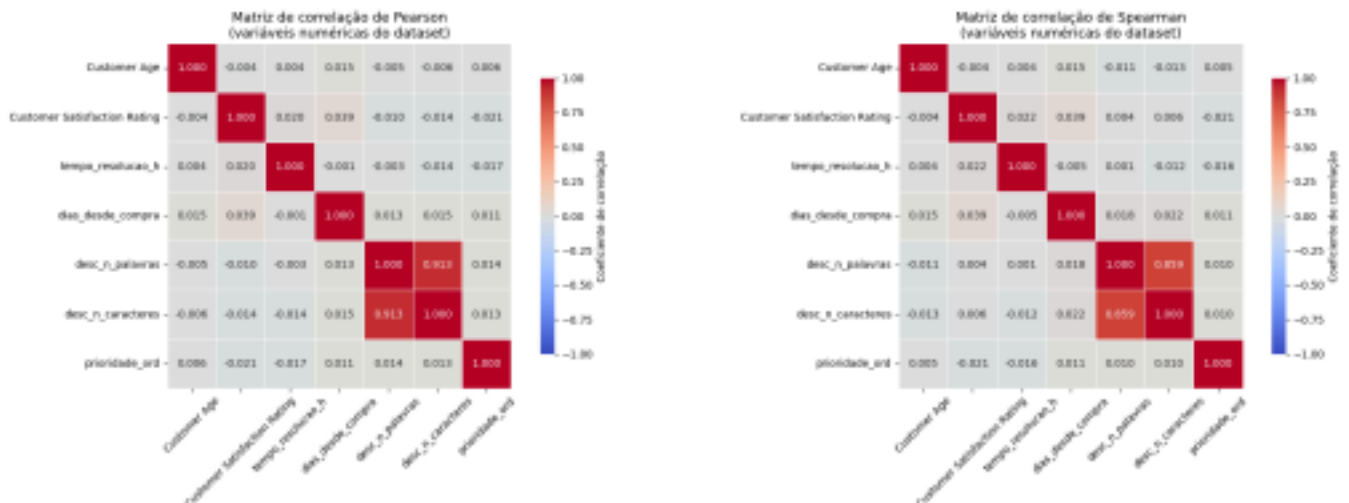
## 1.1 Problema e hipóteses

O objetivo do sistema é identificar automaticamente a intenção de um chamado de suporte a partir do texto do cliente, classificando-o em cinco tipos: problema técnico, cobrança, cancelamento, dúvida sobre produto ou reembolso. A análise buscou verificar se os dados disponíveis são adequados para essa previsão.

As hipóteses levantadas no TP1 foram testadas no TP2:

- **H1:** não existe uma intenção dominante. **Confirmada**, com as classes distribuídas de forma relativamente equilibrada (19,3% a 20,7%).
- **H2:** reembolso e falhas de produto são as intenções mais frequentes. **Não confirmada** ( $p = 0,703$ ).
- **H3:** a urgência percebida não depende do tipo de problema. **Confirmada** pelo teste estatístico ( $p = 0,216$ ).

## 1.3 Heatmap de correlação

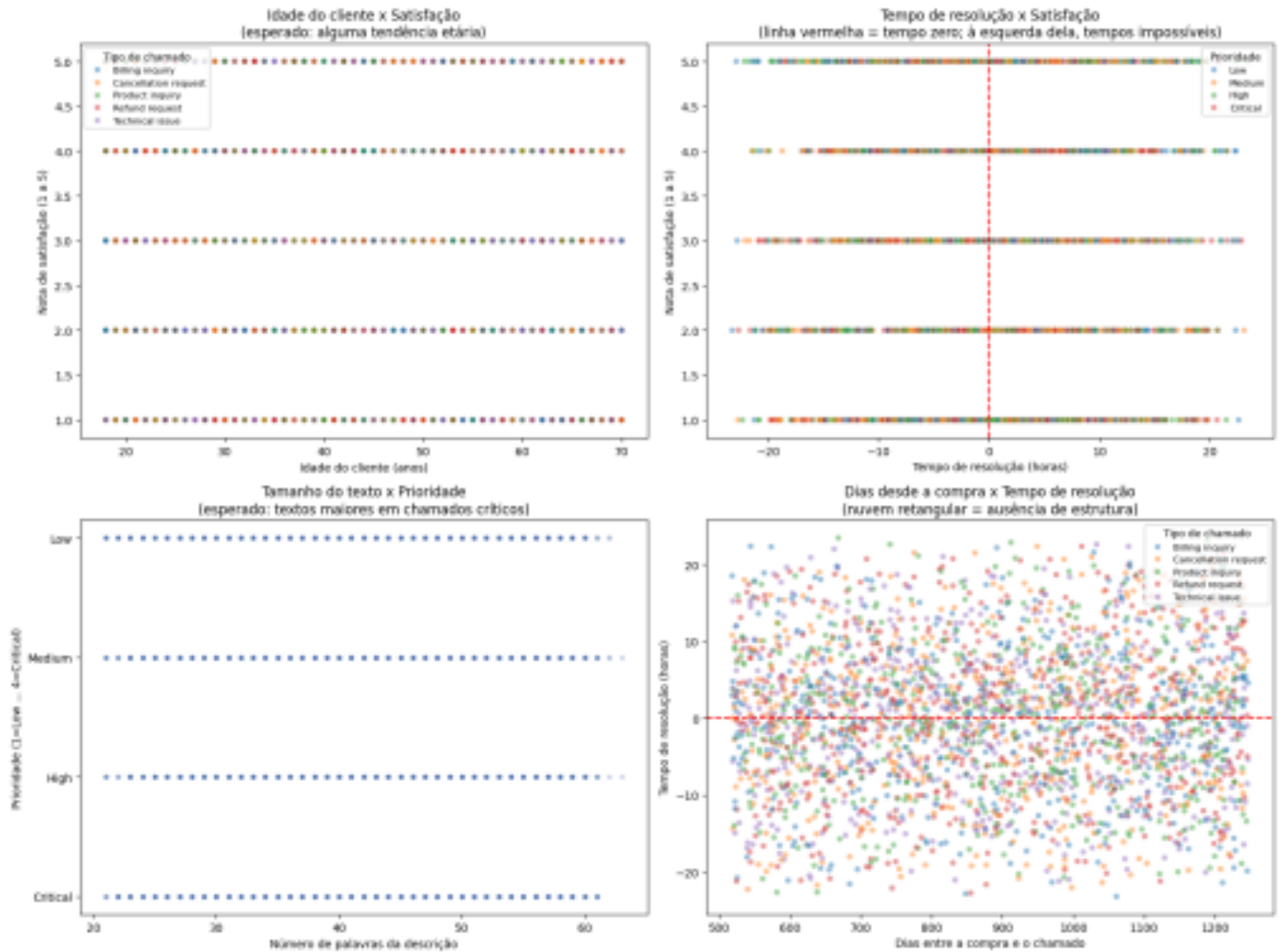


Matrizes de correlação de Pearson e Spearman sobre as sete variáveis numéricas.

O heatmap mostrou uma única correlação forte entre **palavras e caracteres** ( $r = 0,913$ ), algo esperado por serem derivados do mesmo texto. As demais foram muito fracas ( $|r| \approx 0,04$ ), indicando pouca relação entre as variáveis.

Essa falta de relações intuitivas reforça a suspeita de que o **dataset possui características artificiais ou aleatórias**.

## 1.4 Scatter plots



Quatro pares relevantes do ponto de vista de negócio. A linha vermelha marca o tempo zero.

Os quatro gráficos confirmam o resultado do heatmap: **não há padrões claros, agrupamentos ou relações relevantes** entre as variáveis.

Destacam-se dois casos:

- **Tempo de resolução x satisfação:** até tempos negativos apresentam notas variadas, reforçando a inconsistência dos dados.
- **Tamanho do texto x prioridade:** as descrições têm tamanhos semelhantes em todas as prioridades, sem diferença clara entre elas.

No geral, os gráficos reforçam a **falta de relações naturais nos dados**.

## 1.5 Teste de hipótese formal

Foi avaliado se a prioridade do chamado influencia a satisfação do cliente, comparando os grupos Critical e Low.

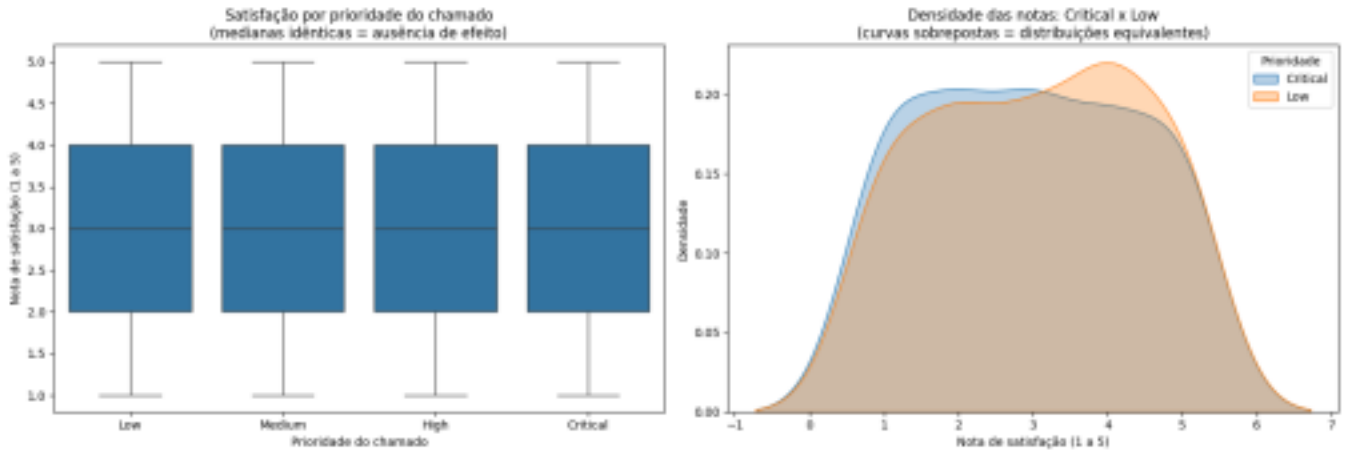
Como os dados não apresentaram distribuição normal, foi utilizado o teste de Mann-Whitney U, com o teste t de Welch como verificação.

Os resultados foram muito semelhantes:

- **Critical:** média 2,959; mediana 3,0.

- **Low:** média 3,053; mediana 3,0.
- **p = 0,216**, acima de 0,05 → **não há diferença estatisticamente significativa**.
- Tamanho de efeito **r = 0,038**, considerado desprezível.

**Conclusão:** a prioridade do chamado praticamente **não está relacionada à satisfação do cliente**, sustentando a Hipótese 3.



Medianas idênticas entre as quatro prioridades e curvas de densidade sobrepostas entre Critical e Low.

O **p-valor de 0,216** indica uma chance de cerca de 22% de observar essa diferença apenas por acaso. Como esse valor é maior que 5%, **não há evidência de diferença significativa** entre as prioridades.

Na prática, as médias foram **2,96 (Critical)** e **3,05 (Low)**, uma diferença de menos de 0,1 ponto.

Isso não prova que os grupos sejam iguais, mas indica que não encontramos diferença nos dados. Junto às correlações praticamente nulas, o resultado reforça a suspeita de que o **dataset foi gerado artificialmente**, com as variáveis sendo pouco relacionadas entre si.

## 1.6 Insights principais

- O dataset é  **sintético**, com pouca relação entre as variáveis e **49,3% de tempos impossíveis**.
- A **prioridade não influencia significativamente a satisfação** ( $p = 0,216$ ;  $r = 0,04$ ).
- O **texto não permite prever a intenção**, com desempenho abaixo do modelo trivial.
- Os **valores ausentes são informativos** e seu uso pode causar *data leakage*.
- A principal descoberta da EDA foi identificar que **o dataset é inadequado para modelagem**, antes de investir em modelos mais complexos.

## 1.7 Limitações

Como o dataset é  **sintético**, os resultados não representam clientes reais. Além disso, o efeito encontrado foi muito pequeno ( $r = 0,04$ ) e a baixa qualidade dos textos (**69,3% idênticos**) limita a classificação por TF-IDF.

## 1.8 Próximos passos (abertura para o modelo de classificação)

A Etapa 3 prevê treinar o classificador. Como demonstrado, com o dataset e a variável-alvo atuais esse modelo terá desempenho de acaso. Há três caminhos:

|                           |                                                                                       |                                                                           |                                                       |
|---------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------|
| A. Trocar o dataset       | (Bitext Customer Support, CLINC150, Banking77)                                        | simulado<br>Sinal real; métricas com significado; comparável à literatura |                                                       |
| B. Trocar a variável-alvo | Prever outra coluna com os metadados existentes                                       | Reaproveita o trabalho feito                                              | A EDA indica correlações nulas também entre metadados |
| C. Manter como scaffold   | Usar o dataset só para exercitar o pipeline, com Adotar base real de intenções modelo | Preserva o foco em segurança<br>Exige refazer a EDA                       | Não entrega classificador funcional                   |

A melhor opção é **trocar o dataset por uma base real**, pois a EDA e a API já podem ser reaproveitadas. Assim, será possível criar um classificador funcional e realizar um pentest mais completo.

O plano é validar primeiro a qualidade do novo dataset, definir um baseline, testar modelos melhores e avaliar com **F1-macro e matriz de confusão**. Depois, o modelo será integrado à rota `/predict`, mantendo a estrutura de segurança já implementada.

## 2. Controles OWASP Top 10 na API

A API do TP1 tinha três rotas, um único usuário `admin` definido no código e nenhuma persistência. O TP2 a reestruturou para tornar os controles verificáveis, mantendo a organização modular original.

### 2.1 Por que a API precisou de múltiplos usuários

O enunciado pede verificação de *ownership* (BOLA) nas rotas que retornam recursos por ID. Com um único usuário e nenhum recurso identificável, esse controle seria impossível de demonstrar. Foram então introduzidos, via `SQLModel`, três usuários ( `admin` , `ana` , `bruno` ) e uma tabela de chamados com `owner_id` , criando o cenário mínimo em que "acessar o recurso de outra pessoa" é uma operação concreta e testável.

### 2.2 A03 - Injection: queries parametrizadas

O projeto não utiliza SQL por concatenação de strings. Todas as consultas passam pelo **`SQLModel/ORM` com parâmetros**, fazendo com que entradas maliciosas sejam tratadas como texto, evitando SQL Injection.

No controle de acesso, cada chamado é verificado pelo **proprietário associado ao token JWT**. Usuários só acessam seus próprios chamados, e recursos de outros usuários retornam **404**, assim como IDs inexistentes, evitando a enumeração de recursos.

### Evidência de execução

Os testes confirmaram os principais controles de segurança: **Ana conseguiu acessar seu próprio chamado (200)**, enquanto **Bruno não conseguiu acessar o chamado dela (404)**. Um chamado inexistente também retorna 404, evitando a enumeração de recursos.

Além disso, o envio indevido do campo `owner_id` foi bloqueado com **HTTP 422**, confirmando a proteção contra *mass assignment*.

### 3. Cabeçalhos de segurança e CORS

Foram implementados diversos **cabeçalhos de segurança**, como HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy e Cache-Control, reduzindo riscos como **clickjacking**, **XSS**, **MIME sniffing** e **vazamento de informações**. A configuração foi validada com `curl`, confirmando a presença dos cabeçalhos esperados.

Durante a verificação, foram encontrados dois problemas que não estavam evidentes no código: o cabeçalho `Server` do Uvicorn continuava sendo enviado e a CSP permissiva ainda era aplicada às rotas de documentação desativadas. Ambos foram corrigidos na configuração do servidor e do middleware.

No **CORS**, foi utilizada uma **allowlist explícita de origens**, definida por variável de ambiente, em vez de `*`. Isso evita que sites não autorizados façam requisições autenticadas e leiam dados da API. Testes confirmaram que apenas origens permitidas recebem o cabeçalho CORS

### 4. Rate limiting no endpoint de autenticação

**Limite adotado:** 5 requisições por minuto, por IP, no POST `/auth/token` (via `slowapi`).

#### 4.1 Por que esse número

O `/auth/token` é o único endereço da API que aceita uma senha e devolve um token válido - alvo óbvio de força bruta e de *password spraying*. Mas há um terceiro risco que costuma passar batido: **derrubar a API pela CPU**. O `bcrypt` é lento de propósito (~100 ms por verificação), e é justamente essa lentidão que protege as senhas. Sem limite, ela vira arma: algumas centenas de tentativas simultâneas de login bastam para saturar o processador. O mesmo controle, portanto, defende as contas e a disponibilidade do serviço.

Para quem usa o sistema de verdade, cinco por minuto é folgado - quem erra a senha tenta duas ou três vezes e parte para a recuperação de conta. Para quem ataca, muda a matemática:

| Cenário           | Tentativas por hora | Tempo para varrer 10.000 senhas comuns |
|-------------------|---------------------|----------------------------------------|
| Sem rate limiting | ~36.000             | ~17 minutos                            |
| Com 5/minuto      | 300                 | ~33 horas                              |

O limite foi definido para tornar ataques de força bruta **mais lentos e detectáveis**, sem prejudicar usuários legítimos. Assim, uma tentativa que levaria minutos passa a exigir **mais de um dia**, dificultando significativamente o ataque.

#### 4.2 Decisões de implementação

O limite de requisições é controlado por **IP**, sem confiar no `X-Forwarded-For` por padrão, evitando que o cliente burle o bloqueio. A resposta **429** é **genérica**, sem informar tentativas restantes ou limites, dificultando ataques de força bruta. As demais rotas possuem limite de **100 requisições por minuto**.

#### Evidência de execução

```
$ for i in 1..8; do curl -X POST /auth/token -d 'username=ana&password=errada'; done
```

tentativa 1 -> HTTP 401

tentativa 2 -> HTTP 401

#### 4.3 O que esse controle ainda não resolve

| Limitação               | Impacto                                            | Encaminhamento                |
|-------------------------|----------------------------------------------------|-------------------------------|
| Contador em memória     | Com N réplicas, o limite efetivo vira $N \times 5$ | Backend Redis                 |
| Contagem por IP         | Botnet ou pool de IPs contorna o limite            | Limite adicional por conta    |
| NAT corporativo         | Usuários legítimos compartilham a cota             | Chave composta (IP + usuário) |
| Sem CAPTCHA progressivo | Não há escalonamento de atrito                     | Desafio após 3 falhas         |

Vale sublinhar a segunda linha: diante de uma botnet, o limite por IP simplesmente não é seguro. O reforço mais importante daqui para frente é somar um **limite por conta** - por mais endereços que o atacante tenha, as tentativas contra o mesmo usuário continuariam caindo no mesmo contador.

## 5. Auditoria com OWASP ZAP

A API foi auditada com o **OWASP ZAP 2.16.1**, usando scan passivo com alta sensibilidade. Foram realizadas duas análises: uma na versão original do TP1 e outra na versão protegida do TP2, permitindo comparar a evolução.

O resultado foi significativo:

- **TP1:** 68 alertas (30 Medium, 17 Low e 21 Informational).
- **TP2:** 18 alertas, todos Informational.
- **Redução: 50 alertas corrigidos**, eliminando todos os riscos Medium e Low.

Entre as principais correções estão **CSP**, proteção contra clickjacking, X-Content-Type-Options e remoção da documentação pública em produção.

Os alertas Informational restantes foram considerados aceitáveis, principalmente por estarem relacionados ao ambiente de teste HTTP e ao funcionamento esperado da autenticação JWT.

**Limitação importante:** o ZAP não consegue validar regras de negócio como **BOLA/ownership**. Por isso, a segurança desses controles foi verificada separadamente por testes automatizados.

**Conclusão:** a auditoria mostrou uma evolução clara da segurança da API, eliminando os principais alertas encontrados na versão inicial.

## 6. Suite de testes automatizados

A API possui **48 testes automatizados** com pytest: 12 funcionais e 36 de segurança. Os principais testes verificam **acesso sem token, BOLA e campos extras**, além de cabeçalhos de segurança, CORS, rate limiting e vazamento de informações sensíveis.

Os testes são executados em um **SQLite em memória**, garantindo isolamento e independência do banco real. Ao final, **todos os 48 testes passaram**, confirmando que os controles de segurança e as funcionalidades esperadas estão funcionando corretamente.

**Resultado:** 48 passed, 1 warning - 50,60s.

## 7. Conclusão

A Etapa 2 terminou com dois resultados de naturezas opostas - e vale separar um do outro.

**Na análise de dados, deu errado. E isso foi útil.** A EDA mostrou, por quatro caminhos independentes que apontam para o mesmo lugar, que o dataset escolhido no TP1 não sustenta o classificador de intenção planejado: correlações nulas entre variáveis distintas, timestamps impossíveis em metade dos chamados fechados, independência estatística entre assunto e tipo, e um baseline que perde para o modelo trivial. Não era o resultado esperado no início - mas descobrir isso agora, com poucas linhas de código e antes de investir em modelagem, é exatamente para o que serve uma EDA. O relatório de EDA traz três alternativas para a Etapa 3, com a recomendação do grupo.

**Na segurança, deu certo.** A API saiu de 30 alertas Medium e 17 Low para zero, com cada controle documentado, coberto por teste automatizado e confirmado por ferramenta externa. E dois defeitos só vieram à tona por causa do scan de verificação - o Server duplicado e a CSP permissiva sobrevivendo em rotas 404. Nenhum dos dois quebrava a aplicação, e nenhum aparecia nos testes; ambos teriam ficado no código se tivéssemos confiado na implementação em vez de auditá-la.

Fica um alerta para a Etapa 4. O controle de ownership - provavelmente o mais importante implementado aqui - **é invisível para o scanner**. Ele existe, funciona e tem sete testes em cima, mas nenhuma ferramenta de scan passivo saberia dizer se o chamado nº 1 é mesmo da Ana. Quem for tentar invadir esta API vai encontrar os cabeçalhos no lugar e o rate limiting ativo; o que vai decidir o resultado do pentest é justamente a camada que o ZAP não enxerga.